

Bilgisayar Programlama

Veri Soyutlama: `enum` ve `typedef`

İçindekiler

1. Numaralandırmalar (`enum`)
 - "Büyülü Sayılar" problemi ve Durum Makineleri
2. Tip Tanımlamaları (`typedef`)
 - Kod Abstraksiyonu ve `<stdint.h>` Standartları
3. İleri Seviye Kombinasyonlar ve İyi Pratikler
 - `typedef enum` kullanımı ve Hata Kodları Sistemi
4. Laboratuvar Çalışmaları ve Özet

BÖLÜM 1

Numaralandırmalar (enum)

"Neden enum?" Motivasyonu

Programlama yaparken genellikle belirli durumları ifade etmek için sayılar kullanırız. Ancak bu sayılar anlamsızdır.

× Büyülü Sayılar (Magic Numbers) Sorunu:

```
1  int motor_durumu = 3;
2
3  // Aylar sonra bu kodu okuyan kişi: 3 ne anlama geliyor?
4  // Hata mı? İleri dönmek mi? Durmak mı?
5  if (motor_durumu == 3) {
6      // ...
7  }
```

✓ Çözüm: Sayılara isim vermek!

```
1  if (motor_durumu == MOTOR_ILERI) {
2      // ...
3  }
```

Temel Sözdizimi (Syntax)

`enum` (enumeration - numaralandırma), birbiriyle ilişkili sabit tam sayı değerlerine isim vermemizi sağlayan veri türüdür.

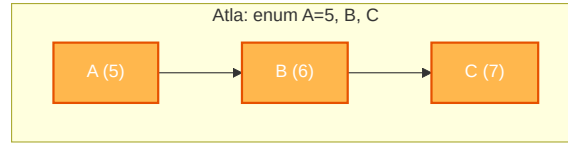
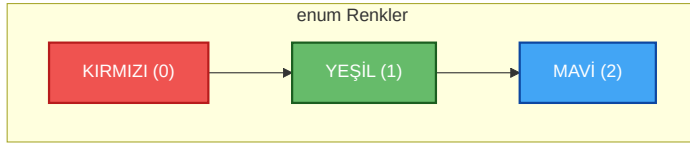
```
1 // enum tanımı
2 enum Gunler {
3     PAZARTESI,
4     SALI,
5     CARSAMBA,
6     PERSEMBE,
7     CUMA,
8     CUMARTESI,
9     PAZAR
10 };
11
12 int main() {
13     enum Gunler bugun = CARSAMBA;
14     return 0;
15 }
```

Arka Plandaki Çalışma Mantığı

C derleyicisi için `enum` içindeki isimler aslında tam sayı (`int`) sabitleridir. Varsayılan olarak ilk eleman `0` değerini alır ve sonrakiler 1'er artar.

```
1  enum Renkler {
2      KIRMIZI, // 0
3      YESIL,   // 1
4      MAVI     // 2
5  };
6
7  int main() {
8      printf("KIRMIZI: %d\n", KIRMIZI); // Çıktı: 0
9      printf("YESIL: %d\n", YESIL);     // Çıktı: 1
10     return 0;
11 }
```

enum Değerlerinin Sayı Doğrusu



💡 Varsayılan: 0'dan başlar, 1'er artar. Araya bir sabit eklenirse oradan devam eder: $A=5 \rightarrow B=6$, $C=7$.

enum Değer Atama

enum içinde değer atanmazsa 0'dan başlar, atanırsa o değerden devam eder:

```
1 enum SensorPini {  
2     SENSOR_1 = 10,  
3     SENSOR_2,      // 11  
4     SENSOR_3,      // 12  
5     OZEL_SENSOR = 20,  
6     SENSOR_4,      // 21  
7     SENSOR_5       // 22  
8 };
```


enum Değer Atama

Neler oluyor?

Adım	Satır	İşlem	Değer	Açıklama
1	<code>SENSOR_1 = 10</code>	Açık atama	10	Başlangıç değeri belirlendi
2	<code>SENSOR_2</code>	Otomatik artış	11	10 + 1
3	<code>SENSOR_3</code>	Otomatik artış	12	11 + 1
4	<code>OZEL_SENSOR = 20</code>	Açık atama	20	Atlama yapıldı (12 → 20)
5	<code>SENSOR_4</code>	Otomatik artış	21	20 + 1
6	<code>SENSOR_5</code>	Otomatik artış	22	21 + 1

⚠ Dikkat: `OZEL_SENSOR = 20` atandığında, C derleyicisi saymaya yeni değerden devam eder. 13-19 arası değerler boşta kalır — bu bazen karışıklığa yol açabilir!

Değer Atama

İstersek numaralandırmaya varsayılan 0'dan değil, kendi belirlediğimiz sayılardan başlamasını söyleyebiliriz.

```
1  enum HataKodlari {
2      BASARILI = 0,
3      HATA_SENSOR = -1,
4      HATA_MOTOR = -2,
5      HATA_BAGLANTI = -3
6  };
7
8  enum BaudRates {
9      BAUD_9600 = 9600,
10     BAUD_115200 = 115200
11  };
```

Mekatronikte haberleşme hızları (Baud rate) veya donanım hata kodlarını tanımlamak için bu yöntem çok sık kullanılır.

Mekatronik Uygulaması: Robot Durum Makinesi

Bir AGV'nin (Otonom Yönlendirmeli Araç) durumlarını (State Machine) yönetelim:

```
1  enum RobotDurumu {  
2      IDLE,          // ● Beklemede  
3      MOVING,        // ● Hareket Halinde  
4      CHARGING,       // ● Şarj Oluyor  
5      EMERGENCY_STOP // ● Acil Durum  
6  };  
7  
8  int main() {  
9      enum RobotDurumu durum = IDLE;  
10  
11     // Sensör engeli algıladı!  
12     durum = EMERGENCY_STOP;  
13  
14     return 0;  
15 }
```

switch-case ile enum Kullanımı

enum tipleri, switch-case yapılarıyla mükemmel bir uyum içinde çalışır. Kodun akışını durum isimleriyle kontrol etmek okunabilirliği en üst seviyeye çıkarır.

```
1  enum RobotDurumu durum = MOVING;
2
3  switch(durum) {
4      case IDLE:
5          printf("🟡 Robot bekliyor.\n");
6          break;
7      case MOVING:
8          printf("🟢 Robot hareket halinde.\n");
9          break;
10     case CHARGING:
11         printf("🔵 Robot sarj ediliyor.\n");
12         break;
13     case EMERGENCY_STOP:
14         printf("🔴 ACIL DURUM! Motorlar durduruldu.\n");
15         break;
16 }
```

enum vs. #define Karşılaştırması ⚠

Sabit tanımlamak için `#define` da kullanılabilir. Neden `enum` tercih etmeliyiz?

× `#define` Yöntemi:

```
1 #define ILERI 1
2 #define GERI 2
```

✓ `enum` Yöntemi:

```
1 enum Yon { ILERI = 1, GERI = 2 };
```

Neden enum?

1. Tip Güvenliği (Type Safety): Derleyici, uyumsuz bir değer atamaya çalıştığınızda uyarı verebilir.
2. Debugger Avantajı: GDB gibi debugger'larda kodu adım adım çalıştırırken, değişkenin değeri `1` yerine `ILERI` olarak görünür. İşinizi çok kolaylaştırır!

BÖLÜM 2

Tip Tanımlamaları (typedef)

typedef Nedir?

`typedef` (Type Definition), var olan bir veri tipine yeni ve anlamlı bir isim (alias - takma ad) vermemizi sağlar.

Amacı:

1. Kodun okunabilirliğini artırmak.
2. Kodun taşınabilirliğini (portability) sağlamak.

```
1 // Sözdizimi: typedef mevcut_tip yeni_isim;
2
3 typedef unsigned char byte;
4 typedef float mesafe_t; // _t soneki (type) yaygın bir standarttır
5
6 int main() {
7     byte piksel_degeri = 255; // Aslında unsigned char
8     mesafe_t lazer_olcum = 12.5; // Aslında float
9
10    return 0;
11 }
```

Neden `typedef` ? Taşınabilirlik (Portability)

C dilinde veri tiplerinin boyutları işlemciden işlemciye (8-bit, 32-bit, 64-bit) değişebilir. Örneğin, bir işlemcide `int` 2 byte (16 bit) iken diğerinde 4 byte (32 bit) olabilir.

Eğer donanım register'larına tam olarak 32 bitlik bir veri yazmanız gerekiyorsa, sadece `int` kullanmak risklidir.

Bu sorunu çözmek için sistem seviyesinde `typedef` kullanılır!

Sistem Seviyesinde Typedef: `<stdint.h>`



C99 standardı ile gelen `<stdint.h>` kütüphanesi, boyutları kesin olarak belli olan tipler sunar. Gömülü yazılım (Embedded C) ve Mekatronikte standart `int` kullanmak yerine her zaman bu tipler tercih edilmelidir!

```
1 #include <stdint.h>
2
3 // Bu tipler aslında arka planda typedef ile tanımlanmıştır:
4 int8_t    sicaklik;    // Kesinlikle 8 bit işaretli (1 byte)
5 uint8_t   pwm_degeri;  // Kesinlikle 8 bit işaretsiz (1 byte, 0-255)
6 int16_t   ivme_x;      // Kesinlikle 16 bit işaretli (2 byte)
7 uint32_t  zaman_milis; // Kesinlikle 32 bit işaretsiz (4 byte)
```

Neden `int` yerine `uint8_t` ? 0 ile 255 arası değer alacak bir PWM sinyali için 4 byte'lık `int` kullanmak %75 bellek israfıdır.

Mekatronik Uygulaması: Birim Tanımlamaları

Fiziksel büyüklükleri kod içerisinde ayırt etmek için `typedef` harikadır. İkisi de `float` olmasına rağmen, Voltaj ve Amper değerlerini karıştırmamızı zorlaştırır.

```
1  typedef float Volt_t;
2  typedef float Amper_t;
3  typedef float Watt_t;
4
5  int main() {
6      Volt_t motor_gerilimi = 12.0;
7      Amper_t cekilen_akim = 1.5;
8
9      Watt_t guc = motor_gerilimi * cekilen_akim;
10
11     printf("Tuketilen Guc: %.2f W\n", guc);
12     return 0;
13 }
```

BÖLÜM 3

İleri Seviye Kombinasyonlar ve İyi Pratikler

typedef enum Kullanımı

Tıpkı `struct` 'larda olduğu gibi, `enum` kullanırken de sürekli `enum` kelimesini yazmamak için `typedef` ile birleştiririz. Gömülü sistemlerdeki en yaygın kullanım şekli budur.

```
1  typedef enum {
2      MOTOR_DUR,
3      MOTOR_ILERI,
4      MOTOR_GERI
5  } MotorState_t;
6
7  int main() {
8      // enum yazmamıza gerek kalmadı
9      MotorState_t sag_motor = MOTOR_ILERI;
10
11      return 0;
12  }
```

Hata Kodları Sistemi Oluşturma

Projeler büyüdükçe, fonksiyonların döndürdüğü hata durumlarını yönetmek zorlaşır. Bir fonksiyon 0 döndürdüğünde "Başarılı" mı, yoksa "0 değerini" mi kastetti?

```
1  typedef enum {
2      SYS_OK = 0,
3      SYS_ERR_TIMEOUT = -1,
4      SYS_ERR_NOT_FOUND = -2,
5      SYS_ERR_I2C_BUS = -3
6  } SystemStatus_t;
7
8  SystemStatus_t Sensor_Baslat(void) {
9      // ... işlemler ...
10     if (baglanti_koptu) {
11         return SYS_ERR_TIMEOUT;
12     }
13     return SYS_OK;
14 }
```

Gömülü Yazılım Standartları (MISRA C)

Mekatronik sistemlerde (özellikle otomotiv, havacılık veya medikal cihazlarda) MISRA C gibi katı kodlama standartları uygulanır.

- Kural: Donanım ile doğrudan etkileşime giren tüm değişkenler için sabit boyutlu türler (`stdint.h`) kullanılmalıdır. Standart `int` kullanımı yasaktır.
- Kural: Durum değişkenleri (State variables) için "Büyülü Sayılar" yerine mutlaka `enum` kullanılmalıdır.

Bu iki araç, kodun sadece sizin için değil, donanım için de güvenilir (Reliable) olmasını sağlar.

Derleme Hatası Kataloğu (Compilation Errors)

Enum ve typedef kullanırken en sık karşılaşılan GCC hataları:

#	Hata Mesajı (GCC)	Sebep	Çözüm
1	<code>expected identifier before numeric constant</code>	<code>enum</code> içinde sayısal sabit kullanılmış	Enum elemanları isim olmalı: <code>SENSOR_1</code> ✓, <code>1</code> ✗
2	<code>redefinition of enumerator</code>	Aynı enum ismi iki kere tanımlanmış	İsimleri benzersiz yapın
3	<code>conflicting types for 'uint8_t'</code>	<code><stdint.h></code> dahil edilmemiş	<code>#include <stdint.h></code> ekleyin
4	<code>unknown type name 'byte'</code>	<code>typedef</code> tanımı görülmeden kullanılmış	<code>typedef</code> 'i kullanımdan önce yapın

Mantık Hatası Kataloğu (Logic Errors)

Derleyici hata vermez ama program doğru çalışmaz:

#	Hata	Belirtisi	Çözüm
1	<code>enum</code> ile <code>switch</code> 'te tüm durumları kontrol etmemek	Bazı durumlar sessizce atlanır	<code>default:</code> ekleyin
2	<code>typedef</code> yapmadan <code>struct</code> / <code>enum</code> kullanmak	Sürekli <code>enum</code> / <code>struct</code> yazmak zorunda kalmak	<code>typedef enum/struct</code> kullanın
3	Aynı ismi iki farklı <code>enum</code> 'da kullanmak	Derleme hatası: redefinition	İsimleri önekle ayırın: <code>ROBOT_IDLE</code> , <code>SENSOR_IDLE</code>
4	<code>enum</code> değerine doğrudan sayı atamak (<code>if(durum == 2)</code>)	"Büyülü Sayı" problemi geri döner	Her zaman enum ismini kullanın

BÖLÜM 4

Laboratuvar Çalışmaları ve Özet

Uygulama 1: Robotik Kol Eksen Durumları

Senaryo: Bir robotik kolun X, Y, Z eksenlerindeki motorlarının durumunu kontrol eden bir sistem yazınız.

```
1  #include <stdio.h>
2  #include <stdint.h>
3
4  typedef enum {
5      EKSEN_HAZIR,
6      EKSEN_HAREKETLI,
7      EKSEN_LIMIT_ASILDI
8  } EksenDurumu_t;
9
10 typedef struct {
```

Beklenen Çıktı:

```
1  Eksen 1: HAZIR
```

- Kullanıcıdan eksen ID ve durum bilgisi alarak çalışan bir program yazın
- 3 eksenli (X, Y, Z) bir dizi oluşturup her birinin durumunu yazdırın

Uygulama 2: Gelişmiş Sensör Paketi

Senaryo: Sıcaklık ve nem değerlerini, sensörün hata durumuyla birlikte döndüren bir yapı (struct) oluşturun.

```
1  typedef enum {  
2      SENSOR_OK,  
3      SENSOR_BAGLANTI_HATASI,  
4      SENSOR_VERI_HATASI  
5  } SensorDurum_t;  
6  
7  typedef struct {  
8      float sicaklik;  
9      float nem;  
10     SensorDurum_t durum;
```

Beklenen Çıktı:

```
1  Sıcaklık: 24.5 C, Nem: 45.2%  
2  Sensor Durumu: OK
```

- Sensör dizisi oluşturun (3 farklı sensör) ve herbirini durum kontrolünden geçirin
- Hatalı sensör durumunda (`SENSOR_BAGLANTI_HATASI`) ekranda kırmızı uyarı gösterin

[{ "Veri Soyutlama": "enum ve typedef" }]

Uygulama 3: Araç Vites Kontrol Sistemi

Senaryo: Bir otonom aracın vites durumunu (PARK, REVERSE, NEUTRAL, DRIVE) tutan bir enum ve mevcut vitesse göre motor gücünü ayarlayan bir fonksiyon yazın.

```
1  typedef enum {
2      GEAR_PARK,
3      GEAR_REVERSE,
4      GEAR_NEUTRAL,
5      GEAR_DRIVE
6  } Vites_t;
7
8  void MotorGucuAyarla(Vites_t mevcut_vites) {
9      switch(mevcut_vites) {
10         case GEAR_PARK:
```

```
1  Vites Testi:
2  Motor gücü 0'a çekildi.
3  İleri hareket için güç veriliyor.
```

- Kullanıcıdan vites girişi alan (1=PARK, 2=DRIVE...) bir menü yazın
- GEAR_REVERSE için geri görüş kamerası mesajı ekleyin

[{ "Veri Soyutlama": "enum ve typedef" }]

Uygulama 4: UART İletişim Paketi

Senaryo: Bir mikrokontrolcünden gönderilecek 5 byte'lık bir veri paketi tanımlayın. Standart `int` yerine `<stdint.h>` tiplerini kullanın.

```
1  #include <stdint.h>
2
3  // Paket yapısı: Başlangıç(1b) + Veri(2b) + Checksum(2b)
4  typedef struct {
5      uint8_t baslangic_bayti;
6      uint16_t sensor_verisi;
7      uint16_t hata_kodu;
8  } UartPaketi_t;
9
10 int main() {
```

1 Paket boyutu: 5 bayt

- Pakete bir `uint32_t zaman_damgasi` ekleyin — boyut kaç bayt olur?
- Birden fazla sensör verisini (`uint16_t`) paketleyip gönderen bir simülasyon yazın

Özet ve Sonraki Ders

Bu Hafta İşlenen Konular:

- Numaralandırmalar (`enum`): Büyülü sayılardan kurtulma, durum makineleri
- Tip Tanımlamaları (`typedef`): Kod okunabilirliği ve taşınabilirlik
- `<stdint.h>` : Sabit boyutlu tiplerle güvenilir gömülü yazılım
- `typedef enum` : En yaygın kullanım şekli
- Hata Kodları Sistemi: Fonksiyon dönüş değerlerini anlamlı hale getirme

Sonraki Ders — Ön Bakış

Açık Kalan Soru:

Farklı tipteki verileri (int, float, char) tek bir çatı altında nasıl toplayabiliriz? Birden fazla sensör verisini birbiriyle ilişkili olarak nasıl saklarız?

👉 Çözüm Gelecek Hafta: Hafta 12 — Yapılar (Structures) ve Birliktelikler (Unions) — Farklı tipteki verileri gruplama, IMU sensör paketleri, bellek hizalama ve padding.

Teşekkürler...

Dr. Öğr. Üyesi Hüseyin YÜCE

Marmara Üniversitesi

Mekatronik Mühendisliği Bölümü